

Elevator Bring Up – Hardware/Software Integration Test Plan

Group 1: Ryan Pratt, Nigel Sinclair, Fergus Page

Purpose

The purpose of this document is to layout the hardware/software integration testing performed on the elevator control system. The goal of the test was to verify that the Raspberry Pi (Rpi) supervisory controller (SC) could communicate correctly with the STM32 elevator controller nodes. The success of this test depends on whether the integrated system could be brought up and working successfully.

This test focused on validating CAN communication, confirming compatibility between nodes, proper implementation of the shared protocol, and identifying any bring-up issues before further development.

System Test Conditions

The system under test consisted of:

- Raspberry Pi supervisory controller running ubuntu
- CAN interface connected as can0
- STM32-based elevator floor control boards
- Existing CAN-based mutual communication protocol
- Elevator control scheduling logic running on the Raspberry Pi
- STM32 firmware implementing the embedded CAN protocol

Only one STM32 board and the Raspberry Pi supervisory controller were reprogrammed during this integration test.

The remaining STM32 boards were intentionally left unchanged so they could retain a known-good working version of the protocol. This reduced risk during bring-up and provided a stable reference for comparison.

Test Scope

In Scope

The scope of this test looked to verify the CAN protocol, and core logical implementation of the FSM controlling the elevator.

This test covered:

- Linux can0 interface setup
- CAN adapter bring-up on the Raspberry Pi
- Communication between the Raspberry Pi supervisory controller and STM32 board
- Verification of the mutual CAN protocol
- Verification of supervisory controller scheduling behaviour
- Firmware/software patching during the test session
- Basic deployment confidence check

Out of Scope

Due to the nature of this test, this test did not fully cover:

- Long-duration reliability testing
- Full safety system validation
- Stress testing with all possible CAN bus traffic conditions
- Formal fault injection testing

Test Setup

The test setup included the Raspberry Pi supervisory controller connected to the CAN network through a PEAK CAN adapter. The CAN adapter was expected to appear on Linux as the can0 interface.

During setup, difficulty was encountered while configuring can0. The root cause was identified as the proprietary PEAK driver claiming the hardware interface instead of allowing the standard Linux SocketCAN driver stack to control the device.

This prevented the supervisory controller software from opening and using the CAN socket correctly.

The issue was corrected by preventing the proprietary PEAK driver from claiming the hardware interface, allowing the standard Linux CAN interface to operate as expected.

Software Deployment Testing

To reduce integration risk, the team reprogrammed exactly:

- One STM32 elevator controller
- The Raspberry Pi supervisory controller

The remaining STM32 boards were not reprogrammed. These boards retained the previously verified working version of the CAN protocol.

This approach allowed the team to compare the updated firmware/software against a known-good baseline and reduced the chance of introducing system-wide communication failures during bring-up.

Test Procedure

Step 1: Prepare CAN Interface

The Raspberry Pi was booted, and the CAN adapter was connected. The Linux CAN interface was checked to confirm whether can0 was available. An initial issue was found where the proprietary PEAK driver claimed the hardware interface. This prevented the expected SocketCAN configuration from functioning correctly. After correcting the driver conflict, can0 was successfully brought up.

Expected result:	Result:
can0 is visible, configurable, and available to the supervisory controller software.	Passed after driver issue was corrected.

Step 2: Reprogram Test Hardware

One STM32 board was flashed with the updated firmware. The Raspberry Pi supervisory controller was also updated with the latest supervisory software. The remaining STM32 boards were left unchanged to preserve a known-good protocol implementation.

Expected result:	Result:
Updated devices boot successfully and join the CAN network.	Passed.

Step 3: Verify CAN Communication

The supervisory controller was started and CAN communication between the Raspberry Pi and STM32 board was observed.

During bring-up, minor bugs were discovered in the STM32 implementation of the mutual CAN protocol. These bugs affected communication behaviour but did not indicate a major design failure.

Expected result:	Result:
The STM32 and Raspberry Pi exchange valid CAN messages according to the expected mutual protocol.	Passed after protocol bugs were patched.

Step 4: Verify Supervisory Scheduler Behaviour

The Raspberry Pi supervisory controller scheduler was tested during system bring-up.

A minor scheduler bug was identified on the Raspberry Pi supervisory controller. The issue was patched during the same test session.

Expected result:	Result:
The supervisory controller schedules and processes elevator control tasks correctly.	Passed after protocol bugs were patched.

Step 5: Bug Regression Check

After patching the STM32 CAN protocol bug and the Raspberry Pi scheduler bug, the system was retested to confirm that the original issues were resolved.

The known-good STM32 boards remained available as a reference point throughout the test.

Expected result:	Result:
The STM32 and Raspberry Pi exchange valid CAN messages according to the expected mutual protocol.	Passed after protocol bugs were patched.

Issues Encountered

The following table shows all issues encountered during the execution of this test plan.

Issue	Location	Severity	Resolution
Can interface setup on Linux	Raspberry Pi/CAN adapter	Medium	Proprietary PEAK driver was prevented from claiming the hardware interface
Proprietary PEAK driver claimed hardware port	Linux driver stack	Medium	Standard Linux CAN interface was restored
Minor mutual CAN protocol bug	STM32 firmware	Low	Patched during bring-up session
Minor scheduler bug	Raspberry Pi Supervisory Controller	Low	Patched during bring-up session

Test Results Summary

This integration test was considered successful.

The main setup issue was related to the PEAK CAN driver configuration on Linux. Once the proprietary driver conflict was corrected, the can0 interface was brought up successfully.

Bring-up testing revealed minor issues in both the STM32 CAN protocol implementation and the Raspberry Pi supervisory scheduler. These bugs were corrected during the same test session, and the system was retested successfully.

Because only one STM32 was reprogrammed, the remaining STM32 boards preserved a known-good protocol version, which reduced deployment risk and helped confirm that the observed issues were isolated to the updated firmware/software.

Conclusion

The elevator hardware/software integration test successfully validated the updated STM32 firmware and Raspberry Pi supervisory controller software in a controlled deployment scenario.

Although setup difficulties were encountered with the Linux can0 interface due to the proprietary PEAK driver claiming the hardware port, the issue was resolved. Bring-up testing also revealed minor bugs in the STM32 mutual CAN protocol and the Raspberry Pi scheduler, both of which were patched during the same session.

The use of a limited deployment strategy, where only one STM32 and the supervisory controller were reprogrammed, helped preserve a known-good baseline and reduced integration risk.

Overall, the test was considered a successful deployment and integration verification.